

WildeCinemas: contexto actual para continuar el proyecto

Actualizado: 6 de octubre de 2026. Documento para estudiar, trasladar el trabajo a otra computadora o compartir el contexto con otro asistente.

1. Objetivo del usuario

El proyecto es un trabajo de Programación IV. El usuario necesita continuar su desarrollo y defenderlo en un examen oral: entender qué hace cada componente y servicio, quién depende de quién, cómo se comunican y dónde se ejecutan las reglas.

Prefiere explicaciones breves, concretas y comentarios básicos dentro del código. Solicitó trabajar con rapidez, conservar el funcionamiento existente y separar servicios cuando concentran demasiadas responsabilidades. No quiere un calendario desplegable para introducir fechas: quiere editar día, mes y año por segmentos.

2. Proyecto y estado local

- Nombre: WildeCinemas, aplicación de cine.
- Carpeta de esta PC: C:/Users/uld001/Desktop/Programacion-4-TP-N-1.
- Rama observada: main. Último commit observado: e2b9cff, "detalles y documentacion".
- Hay cambios locales y archivos nuevos SIN COMMIT. No se realizó commit ni push de esta tanda de cambios.
- Tecnologías declaradas: Angular 22.1, TypeScript 6.0, Supabase JS 2.116, RxJS, jsPDF, qrcode y Vitest. Consultar package.json para las versiones instaladas y permitidas.
- Angular usa componentes standalone, Router, signals y computed. No todos los servicios son simplemente llamadas a Supabase: también adaptan datos y aplican lógica de interfaz.

El contexto ayuda a continuar, pero no reemplaza los archivos del proyecto ni la base de datos. En otra PC se necesitan los cambios sin commit y los archivos nuevos, además de la configuración correspondiente.

3. Arquitectura y comunicación

Flujo general: usuario -> componente -> servicio Angular -> cliente Supabase compartido -> Auth, tablas, Storage o RPC -> respuesta -> servicio adapta datos -> componente actualiza la vista.

AuthService crea una única instancia de Supabase desde los archivos de src/environments y expone el getter client. Los servicios de datos reutilizan esa instancia. PerfilService recibe el cliente y el usuario como argumentos; SesionService no necesita conectarse a Supabase.

Existen accesos directos desde componentes: AdminUsuarios invoca una Edge Function; AdminReportes invoca RPC de reportes; ValidarQr invoca validar_qr_orden; Butacas limpia el canal Realtime. No afirmar que absolutamente toda consulta pasa por un servicio específico de dominio.

La autorización real y las reglas transaccionales se ejecutan en Supabase mediante RLS, RPC y triggers. Los guards controlan navegación; la directiva y los bloques @if controlan lo que se muestra.

4. Recorrido de la aplicación

1. Inicio muestra cartelera, próximos estrenos, ranking de ventas y promoción vigente. Ofrece búsqueda y filtros.
2. El detalle presenta una película, funciones y reseñas.
3. El usuario elige una función y entra a Butacas. Puede comprar como invitado o con sesión, sujeto a las reglas de la compra.

4. Elegir una butaca intenta reservarla temporalmente. Se actualiza la disponibilidad por Realtime y consultas periódicas.
5. Comprar llama a una RPC que valida y confirma la operación. El servidor calcula descuentos, importes y puntos.
6. Ticket muestra el comprobante, QR y descarga PDF.
7. Mis películas muestra historial personal, alertas, puntos, crédito y opciones de cancelación.
8. Empleados y administradores pueden entrar a Validar QR. El servidor comprueba el código y su uso.
9. Admin contiene películas, salas/butacas, funciones, configuración, usuarios y reportes. Sus módulos son pestañas/componentes dentro de AdminComponent, no rutas hijas independientes.

5. Servicios actuales y responsabilidad

- AuthService: autentica, mantiene el usuario y coordina perfil y duración. Lo usan formularios, guards, layout y servicios que necesitan client.
- PerfilService: consulta perfiles, transforma datos y mantiene currentProfile. Lo utiliza AuthService.
- SesionService: controla el plazo máximo de sesión en el navegador. Lo configura y utiliza AuthService.
- PeliculasService: consultas y operaciones sobre películas. Se usa en cartelera, detalle y administración.
- PeliculasCrudService: estado y acciones del CRUD administrativo de películas; coordina carga, edición, guardado y borrado.
- SalasFuncionesService: salas, funciones y precio global. Invoca las RPC de programación; se usa en detalle y módulos administrativos. Es el nombre actual: no volver a ProgramacionService sin revisar el proyecto.
- ButacasService: asientos, reservas, ocupación y canal Realtime. Se usa en la pantalla de selección.
- ComprasService: envía la compra a la RPC y adapta la respuesta a ComprobanteCompra. Se usa desde Butacas.
- ReseniasService: consulta resúmenes/reseñas y guarda la calificación del usuario. Se usa en Inicio y ReseniasPelicula.
- ExperienciaClienteService: ranking, alertas, historial y cancelaciones. Se usa principalmente en Inicio y MisPeliculas.
- CuponesService: consulta y administra promociones. Se usa en configuración y para mostrar la promoción de Inicio.
- ImagenesService: valida y sube pósters a Storage. Se usa en FormularioPelicula.

6. División reciente de AuthService

Antes tenía 405 líneas con comentarios y espacios. Se separaron perfil y duración de sesión para que cada responsabilidad sea fácil de estudiar. AuthService conserva la interfaz que ya usan los componentes y guards.

AuthService inyecta PerfilService y SesionService. Para evitar dependencia circular, SesionService recibe dos funciones mediante configurar: obtener el UUID actual y pedir a AuthService el cierre local. PerfilService no inyecta AuthService: recibe client y user en cargar.

AuthService: todas sus funciones

- constructor: crea Supabase, configura SesionService e inicia initAuth.
- whenReady: espera la inicialización y verifica vencimiento; importante para guards al actualizar la página.
- login: recibe credenciales, llama a signInWithPassword, inicia el reloj, guarda usuario y carga perfil; devuelve success/error.
- register: llama a signUp con metadata de cliente. El trigger crea perfiles. Solo carga sesión si Supabase devuelve una sesión activa.
- logout: llama a signOut y limpia usuario, perfil y reloj; devuelve success/error.
- refreshCurrentProfile: recarga puntos, saldo y demás datos del usuario actual.
- client: getter que entrega la instancia compartida de Supabase.
- initAuth, privada: recupera la sesión con getSession y escucha onAuthStateChange.

- `mensajeErrorRegistro`, privada: adapta ciertos errores a mensajes comprensibles.
- `cerrarSesionLocal`, privada: cierra la sesión vencida con scope local, limpia usuario/perfil y navega automáticamente al inicio.

Expone `currentUser`, `currentProfile`, `isLoading`, `isLoggedIn`, `userRole`, `isAdmin`, `isEmpleado` e `isCliente`. `currentProfile` es la MISMA señal que actualiza `PerfilService`.

PerfilService: todas sus funciones

- `cargar(client, user)`: consulta perfiles por UUID y adapta campos `snake_case` a `camelCase`, números y valores de respaldo. Actualiza `currentProfile`.
- `limpiar`: borra el perfil e invalida las consultas pendientes.

Se corrigió un fallo comprobado: una respuesta que llegaba después del logout podía restaurar el perfil anterior. Un contador `versionConsulta` invalida respuestas viejas al limpiar o iniciar otra consulta. También evita que una consulta anterior sobrescriba una más reciente.

Se conserva el respaldo de `user_metadata`, incluido el rol, si no se obtiene el perfil. Esto influye en la interfaz; NO sustituye la autorización comprobada por PostgreSQL.

SesionService: todas sus funciones

- `configurar`: recibe las funciones de consulta/cierre y registra `visibilitychange`.
- `iniciar`: guarda inicio y programa el vencimiento, sin reiniciar una marca existente.
- `recuperar`: devuelve si una marca está vigente; `SIGNED_IN` puede crear una marca faltante, renovar el token no reinicia el tiempo.
- `verificar`: consulta al usuario y pide cerrar si venció.
- `cerrar`: evita cierres simultáneos, ejecuta el cierre recibido y limpia el reloj.
- `limpiar`: cancela el temporizador y elimina la marca guardada.
- `guardarInicio`, privada: guarda `userId` e inicio en `localStorage`.
- `leerInicio`, privada: valida la marca y devuelve null si no corresponde.
- `vencida`, privada: compara el tiempo transcurrido con el máximo.
- `programar`, privada: programa la comprobación con el tiempo restante.

7. Sesión: valor final y redirección

El valor FINAL es $7 * 60 * 1000$ en `SesionService.DURACION_MAXIMA_MS`. La clave de `localStorage` sigue siendo `wildecinemas.session.startedAt` y guarda `userId` e inicio. Actualizar o renovar el token no extiende el plazo.

El temporizador, `whenReady` y `visibilitychange` verifican vencimiento. El cierre usa `signOut({scope: 'local'})`. Este límite está implementado en el navegador; no afirmar que se configuró un máximo de sesión en el servidor Supabase. La caducidad de un access token y la duración total de una sesión son conceptos distintos.

Para la prueba real se bajó temporalmente a 30 segundos, con autorización del usuario, y después se RESTAURÓ a siete minutos. No dejar 30 segundos como configuración definitiva.

Se observó que originalmente la pantalla abierta permanecía visible después del cierre. El usuario pidió corregirlo: ahora `cerrarSesionLocal` limpia el estado y llama a `router.navigate([''])` dentro de `finally`. La redirección también se ejecuta si Supabase devuelve un error de cierre. Esta última redirección tiene prueba automatizada; no se volvió a realizar un login real después de agregarla.

8. Guards y directiva documentados

- `authGuard` protege `/mis-peliculas`: exige usuario autenticado; si falta, manda al login con `redirect` igual al destino original. Actualmente `AuthComponent` no consume ese parámetro al completar login.

- adminGuard protege /admin: exige isAdmin; los demás van al inicio.
- staffGuard protege /validar-qr: permite admin o empleado. Cliente va al inicio; visitante va al login.
- Todos esperan whenReady antes de consultar el estado. Se documentaron authGuard y staffGuard con comentarios básicos; adminGuard ya tenía comentarios.
- SoloAdminDirective es estructural. Su único uso actual está en el enlace Panel de Control del layout con *soloAdmin. Usa TemplateRef, ViewContainerRef y effect para crear/quitar la vista según isAdmin; hasView evita duplicarla. También se documentó.
- El enlace Validar QR se muestra mediante @if (isEmpleado() || isAdmin()) en AppLayout; no usa una directiva personalizada.

9. Formularios, fechas y pipes

Registro usa formularios reactivos (FormBuilder, FormGroup, Validators). Varias pantallas administrativas y formularios simples usan FormsModule/NgModel. Angular también tiene Signal Forms, pero el proyecto no los implementó. Los signals de estado no equivalen a usar Signal Forms.

SelectorFechaComponent implementa ControlValueAccessor: permite editar DD/MM/AAAA por segmentos, filtra caracteres no numéricos y comunica una fecha ISO completa y válida o una cadena vacía. Se integra con formularios reactivos y NgModel. No abre calendario; técnicamente sus segmentos son inputs de texto.

Se implementó en los lugares faltantes de programación y reportes. AdminFunciones separa fecha y hora, y valida inicio futuro. Reportes valida fechas completas y rango no invertido. La revisión no encontró type=date ni datetime-local en src/app.

FechaArgentinaPipe formatea fechas y protege la presentación ante valores inválidos; para fechas sin hora evita el cambio de día por zona horaria. Se reutilizó en Ticket, MisPelículas, reportes y mensajes de funciones. MonedaArgentinaPipe presenta ARS. DecimalPipe se usa para promedios. Los pipes no reemplazan las validaciones del formulario ni del servidor.

10. Usuarios desde administración

Se simplificó la creación a cinco campos: nombre, apellido, correo, contraseña y rol. Se eliminaron los requisitos de nacimiento, sangre, ojos y vacaciones en esta pantalla y en la Edge Function; esto no elimina los campos del registro de clientes.

El componente valida campos vacíos o con espacios, email, contraseña mínima de ocho caracteres y rol permitido. Usa NgForm y evita doble envío. La función crear-usuario-privilegiado verifica la identidad del solicitante y su rol admin, valida los cinco campos, crea Auth y asigna el rol del perfil desde el servidor. Si falla la asignación, intenta retirar la cuenta recién creada. Registra auditoría sin contraseña.

Archivo local: supabase/functions/crear-usuario-privilegiado/index.ts. La versión de cinco campos se desplegó y se verificó en Supabase durante la revisión.

11. Cupones y primera compra

Se agregó AdminCuponesComponent dentro de Configuración y CuponesService. El administrador puede cambiar el porcentaje de primera compra y crear/editar cupones activos, con restricción opcional para mayores de 50.

La consigna final es edad, no condición de jubilado. La regla SQL actual usa años cumplidos > 50: habilita desde los 51 años. Un cupón restringido necesita sesión y fecha de nacimiento registrada.

Los porcentajes se validan como finitos, con hasta dos decimales. Primera compra permite cero para desactivar el beneficio; un cupón manual requiere un porcentaje positivo, hasta 100. Los códigos se normalizan a mayúsculas, admiten 3 a 30 caracteres de letras/números/guiones y PRIMERA_COMPRA está reservado. La edición se hace por ID para poder cambiar el código sin crear otro cupón.

ComprasService envía p_codigo_cupon; la RPC calcula el descuento y devuelve porcentaje_descuento. No se acumulan descuentos: se elige el mayor entre primera compra y cupón elegible. Ticket y PDF muestran el porcentaje real; Inicio lee la configuración. Butacas muestra el total antes de descuentos y recarga el perfil tras una compra exitosa.

SQL local: supabase/sql/cupones-configurables.sql. Se comprobó que ya estaba aplicado en el servidor; no atribuir una nueva ejecución de ese script a la última refactorización.

12. Imágenes desde la PC

FormularioPelicula mantiene la URL y agrega selección de archivo. ImagenesService permite JPG, PNG y WebP, máximo 5 MB, solo para admin. Genera una ruta con UUID, sube sin sobrescribir y completa la URL pública. El formulario bloquea el guardado mientras sube.

El bucket peliculas-imagenes y sus políticas se crearon/verificaron mediante supabase/sql/imagenes-y-auditoria-descuentos.sql. La lectura de pósters es pública; la subida exige rol admin. No se implementó limpieza automática de imágenes subidas que luego se abandonan.

13. Salas, programación y separación de funciones

Se incorporó el borrado dentro de Salas y Butacas. La solución persistente usa eliminar_sala_sin_funciones: verifica admin, bloquea la sala, rechaza salas con funciones y elimina butacas/sala atómicamente. El servicio exige respuesta true. El usuario indicó que ejecutó el SQL. Archivo: supabase/sql/eliminar-sala-sin-funciones.sql.

crear_funcion_con_sala_automatica asigna una sala compatible con formato e idioma y con disponibilidad; el fin depende de la duración de la película. Hay bloqueo transaccional para evitar asignaciones concurrentes incompatibles. La lógica importante está en PostgreSQL y SalasFuncionesService la reutiliza mediante RPC.

La separación mínima es de 30 minutos entre funciones de una misma sala. Se controla al asignar y mediante trigger en escritura. Exactamente 30 minutos está permitido: si una función termina a las 20:00, otra puede comenzar a las 20:30.

La generación de sala observada en el servidor produce 518 asientos: 420 normales, 84 VIP y 14 accesibles; la fila K no se utiliza. No confundir el texto comercial de 20 filas con la cantidad efectiva de filas generadas.

14. Butacas, Realtime y compra

La reserva dura cinco minutos. El cliente envía un token y el servidor guarda su hash. La RPC comprueba pertenencia a la sala, asiento no vendido y reserva no ocupada por otro token vigente.

butacas_ocupadas_funcion incluye asientos pagados y reservas vigentes. El canal postgres_changes escucha reservas_butacas filtradas por función y vuelve a consultar. Hay un respaldo de consulta cada 15 segundos. Al destruir la pantalla se limpia canal/timer y se intenta liberar la selección. Se verificó la inclusión de la tabla en supabase_realtime; no se hizo una prueba simultánea real con dos navegadores.

Comprar valida 1 a 8 asientos, duplicados, reservas, clasificación y preventa. Aplica precio VIP, descuento y puntos en el servidor. La RPC marca PAGADA; no hay una pasarela real de cobro. Cancelar una compra elegible acredita saldo y revierte puntos; no es una devolución a tarjeta. El crédito todavía no se usa como medio de pago en la compra actual.

15. Experiencia del usuario y alertas

Ranking muestra las tres películas con más entradas pagadas vendidas, no las mejor calificadas. Las reseñas se guardan por usuario/película. MisPeliculas consulta historial, puntos, crédito y alertas.

Activar una alerta requiere sesión y hace upsert por usuario/película. En Mis películas, una alerta figura disponible si la película está EN_CARTELERA o tiene una función futura. No hay email, push ni proceso que marque notificada=true en este flujo. El texto "Ya hay funciones disponibles" puede ser más específico que la condición real.

El Candy Bar administrativo sigue siendo una pantalla incompleta. Existen tablas, QR y reportes relacionados, pero no se completó la experiencia de canje de puntos/productos. No presentar esas funciones como terminadas.

16. Auditoría, reportes y descargas

Auditoría combina triggers de cambios de tablas con logs explícitos de algunas operaciones. Guarda operación, entidad, actor, fecha y datos JSON limitados; no registra cada lectura/clic ni un diff completo de antes/después.

Se corrigió `log_actividad.usuario_id` para permitir NULL: las operaciones sin `auth.uid` se muestran como Sistema. Se agregaron triggers para los nuevos cupones/configuración. El cambio se aplicó en Supabase y queda en `imagenes-y-auditoria-descuentos.sql`.

AdminReportes consulta cinco RPC en paralelo y filtra reportes por fecha; auditoría reciente tiene su propio alcance. CSV diario usa separador punto y coma, comillas, BOM, Blob y ObjectURL. El PDF de reportes usa impresión del navegador/CSS. Ticket usa jsPDF y qrcode; son mecanismos distintos. Los gráficos de reportes son CSS.

El QR es un código del servidor. `validar_qr_orden` exige personal autorizado, orden pagada y control de uso único por tipo; registra la validación. Los comprobantes de Ticket dependen de datos de navegación/sessionStorage, no de una recuperación completa desde cualquier navegador.

17. Cambios anteriores y documentación

Se actualizó README y se retiró el enlace del programa de puntos del footer. Se estudió el flujo y se explicó la separación de programación; el servicio actual se llama SalasFuncionesService. Para la explicación completa consultar `docs/guia-profesor.md`.

Otros documentos: `docs/verificacion-profesor.md` contiene la revisión anterior; `docs/verificacion-auth-dividido.md` contiene las pruebas de autenticación y la corrección posterior de redirección. Sus resultados históricos no deben confundirse con la última batería de pruebas presente en el proyecto.

Este contexto consolida el estado visible y los cambios revisados; no es una transcripción literal de todas las conversaciones ni demuestra por sí solo el estado remoto de Supabase.

18. Verificación más reciente

- Última ejecución: `npm test -- --watch=false`, 65 pruebas aprobadas en 8 archivos, salida 0. Incluye la nueva redirección al inicio al vencer y el caso de error de cierre.
- Se ejecutó compilación de desarrollo correctamente antes de la última redirección. El cambio de redirección también fue compilado por Angular al ejecutar las pruebas. No afirmar que se verificó una compilación de producción.
- Prueba real: el usuario inició sesión como admin; se verificaron menú/rol, recuperación al actualizar y acceso a Validar QR y Mis películas sin errores de consola observados.
- Prueba real del vencimiento: plazo temporal de 30 segundos, cierre y bloqueo de acceso posterior confirmados. La redirección automática agregada después se comprobó con pruebas automatizadas.
- Registro y roles empleado/cliente se probaron con respuestas simuladas; no se crearon cuentas reales para esa batería.
- Se reprodujo y corrigió la respuesta tardía de perfil después del logout; también se prueba el orden inverso de dos consultas.
- Anteriormente se ejecutaron cinco comprobaciones reales de alertas en SQL, con BEGIN/ROLLBACK y datos temporales. No quedaron filas de prueba.
- Vitest usa un worker de threads en `vitest.config.mjs` para evitar problemas de arranque de procesos en este entorno Windows.

Los archivos de pruebas nuevos están en los tres servicios de auth, AdminUsuarios, Inicio y SelectorFecha. Las pruebas existentes también se ejecutaron. No inventar cobertura sobre flujos que no se probaron.

19. Para continuar desde otra PC

1. Llevar el proyecto completo actualizado, incluidos archivos nuevos y cambios sin commit. Si se usa Git, revisar y guardar los cambios antes de sincronizar; este chat no hizo commit/push.
2. Llevar este Markdown y el PDF. Para otro asistente, adjuntar el Markdown y permitirle leer el repositorio.
3. Instalar dependencias con `npm install`; iniciar con `npm start`. La prueba de esta PC usa `127.0.0.1:4201`; la ejecución normal suele usar `localhost:4200`.
4. Recordar que `localhost` y `127.0.0.1`, y los distintos puertos, no comparten necesariamente el mismo almacenamiento de sesión. Una sesión local tampoco se traslada con el código.
5. Usar el mismo proyecto Supabase y la configuración correspondiente, sin publicar credenciales sensibles ni colocar `service_role` en Angular.
6. No recrear SQL a ciegas: el repositorio no tiene todos los scripts históricos. Verificar primero tablas, RPC, políticas y funciones existentes en el servidor.

Tener la misma cuenta de ChatGPT/Codex no asegura que un chat local aparezca automáticamente en otra PC. El acceso remoto a la computadora de origen es otra opción, cuando está configurado y esa computadora permanece disponible.

20. Pendientes y prioridades sugeridas

- Explicar al usuario los servicios y componentes con el mismo nivel básico, siguiendo el código real.
- Si se desea una comprobación adicional, repetir el vencimiento real con redirección automática en la versión final; restaurar siempre el plazo a siete minutos.
- Completar pruebas reales de cliente/empleador, registro y concurrencia de reservas cuando haya cuentas de prueba.
- Revisar el uso del parámetro `redirect` en login y el respaldo de rol desde metadata antes de cambios de seguridad.
- Revisar el mensaje de disponibilidad de alertas, fechas opcionales incompletas y limpieza de imágenes abandonadas.
- Completar Candy Bar/canje y uso de crédito si la consigna lo requiere; actualmente no están completos.
- Obtener un inventario/exportación versionada del backend para poder reproducir el proyecto sin depender de SQL histórico ausente.
- Actualizar este contexto cuando cambie el proyecto. Preservar cambios del usuario; no reiniciar ni sobrescribir archivos sin revisarlos.

21. Mensaje para pegar en otro chat o asistente

Estoy continuando WildeCinemas, un proyecto Angular standalone con Supabase. Leé `docs/contexto-actual-proyecto.md`, `docs/guia-profesor.md` y los archivos actuales antes de modificar. Hay cambios sin commit que debés preservar. AuthService ahora coordina PerfilService y SesionService; el máximo local es siete minutos y al vencer redirige al inicio. Se agregaron cupones configurables, subida de pósters, validación de usuarios admin y fechas por segmentos. Necesito entender y defender cada función en un oral: explicaciones breves, comentarios básicos y comunicación entre componentes/servicios. No inventes funciones terminadas ni pruebas realizadas; comprobá el estado actual del código y del backend cuando corresponda.